

# ObjectSelector User Guide

2026-08-15

# ObjectSelector User Guide

## 概要

`objectselector` は、Qt の `QObject` / `QWidget` を探索して選択するためのライブラリーです。

現在の配布バージョンは `VERSION` ファイルで管理する `1.0.0` です。

主に次の用途を想定しています。

- 画面上の `QWidget` を直接指して選ぶ
- オブジェクトツリーから選ぶ
- 選択対象をハイライト表示する

このモジュールは「選択」を担当します。

選択した `QObject` を表示・編集する機能は含みません。

配布用 `.zip` にはビルド済みバイナリーではなく、ソース一式が入っています。

## 含まれるもの

- 共有ライブラリー `objectselector`
- デモアプリケーション
  - `objectpicker_demo`
  - `objecttree_demo`
- テスト
  - `objectpicker_test`
  - `objecthighlighter_test`

主なクラスは次です。

- `ObjectPicker`
- `ObjectTreeWidget`
- `ObjectTreeDialog`
- `ObjectHighlighter`

## ビルド前の準備

ビルドには次が必要です。

- CMake 3.16 以上
- C++17 対応コンパイラ
- Qt 6
- Qt6 の次のモジュール
  - `Core`
  - `Gui`
  - `Widgets`
  - `Test`

Qt のインストール場所が標準パスにない場合は、`CMAKE_PREFIX_PATH` を指定してください。

例:

```
cmake -S . -B build \  
-DCMAKE_PREFIX_PATH=/usr/local/qt/Qt/6.11.0/macos \  
-DBUILD_TESTING=ON
```

## ディレクトリー構成

```
objectselector/  
  CMakeLists.txt  
  USER_GUIDE.md  
  src/  
  tests/  
  examples/
```

- `src/` ライブラリー本体です
- `tests/` 単体テストです
- `examples/` デモアプリケーションです

## ビルド方法

```
cmake -S . -B build -DBUILD_TESTING=ON  
cmake --build build
```

テストをビルドしない場合は `-DBUILD_TESTING=ON` を省略できます。

## インストール

```
cmake --install build
```

既定のインストール先は、`ICPP_PREFIX` が指定されている場合はその値、そうでない場合は `PATH` 上の `icpp` 実行ファイルの親ディレクトリーです。いずれも利用できない場合は `CMake` の既定値を使用します。

任意の場所へインストールするには、構成時に `CMAKE_INSTALL_PREFIX` を指定してください。

```
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/path/to/install  
cmake --build build  
cmake --install build
```

インストール後の配置は次のとおりです。

- `bin/` : デモアプリケーション
- `include/` : 公開ヘッダ
- `lib/` : 共有ライブラリー

## source code release zip

配布 ZIP は source code release です。ビルド成果物、インストール済みファイル、Git 管理情報は含みません。

```
unzip objectselector-1.0.0-source.zip  
cd objectselector-1.0.0
```

```
cmake -S . -B build -DBUILD_TESTING=ON
cmake --build build
```

リリースを作成する場合は、リポジトリのルートで次を実行します。

```
./release.sh
```

生成される ZIP は `release/objectselector-<VERSION>-source.zip` です。 `release.sh` は一時 `staging directory` で作成し、ZIP の整合性検査も行います。

## 生成物

ターゲット名は `objectselector` です。

生成ライブラリー名は OS に応じて次になります。

- macOS: `objectselector.dylib`
- Linux: `objectselector.so`
- Windows: `objectselector.dll`

デモ実行ファイルは次です。

- `objectpicker_demo`
- `objecttree_demo`

## テスト実行

```
ctest --test-dir build --output-on-failure
```

## デモの実行

```
./build/objectpicker_demo
./build/objecttree_demo
```

## ObjectPicker の使い方

`ObjectPicker` は、表示中のルート `QWidget` を渡して使います。

```
auto *picker = new ObjectPicker(this);

connect(picker, &ObjectPicker::objectPicked,
        this, [this](QObject *targetObject) {
    // 選択結果を利用する
});

picker->start(rootWidget);
```

### 基本動作

- `start(rootWidget)` でピッカーモードに入ります
- マウス下の候補 `QWidget` は一時ハイライトされます

- 左クリックでその対象を確定します
- `Esc` または右クリックでキャンセルできます
- ピッカーモード中は対象領域を通常操作できません

### キャンセル時の挙動

- `Esc` または右クリックで終了した場合、今回のピックでは何も選択しなかったものとして扱います
- `objectPicked` は `emit` されません
- `selectedObject()` は `nullptr` になります

### 確定選択の管理

`ObjectPicker` は外側から確定選択を受け取れます。

- `setSelectedObject(QObject *)` 外側から現在選択として扱いたい対象を設定します
- `selectedObject()` 現在の確定選択を取得します

典型的には、ツリーで選んだ対象や別 UI の選択結果を `setSelectedObject()` で反映します。

### シグナル

- `objectPicked(QObject *targetObject)` 左クリックで選択が確定したときに `emit` されます
- `hoveredObjectChanged(QObject *targetObject)` ホバー中の候補が変わったときに `emit` されます
- `activeChanged(bool active)` ピッカーモードの開始・終了時に `emit` されます

## ObjectTreeWidget の使い方

`ObjectTreeWidget` は `QObject` の親子関係をツリーで表示します。

```
auto *treeWidget = new ObjectTreeWidget(this);
treeWidget->setRootObject(rootObject);
treeWidget->setCurrentObject(currentObject);

connect(treeWidget, &ObjectTreeWidget::objectActivated,
         this, [this](QObject *targetObject) {
             // 選択結果を利用する
         });
```

### 主な機能

- ルートオブジェクト配下のオブジェクト一覧を表示します
- オブジェクト名とクラス名でフィルタできます
- 現在選択中の対象へスクロールします
- パンくず表示で現在位置を確認できます

## ObjectTreeDialog の使い方

`ObjectTreeDialog` は、`ObjectTreeWidget` を単独ダイアログとして使いたい場合に使います。

```
auto *treeDialog = new ObjectTreeDialog(this);
treeDialog->setRootObject(rootObject);
treeDialog->setCurrentObject(currentObject);
treeDialog->show();
```

選択通知は内包している `ObjectTreeWidget` から受け取ります。

```
connect(treeDialog->objectTreeWidget(), &ObjectTreeWidget::objectActivated,
        this, [this](QObject *targetObject) {
            // 選択結果を利用する
        });
```

## ObjectHighlighter の使い方

ObjectHighlighter は、任意の QObject をハイライト表示します。

```
auto *highlighter = new ObjectHighlighter(this);
highlighter->setTargetObject(targetObject);
```

- QWidget は矩形塗りつぶしで表示します
- QLayout は外形線で表示します
- clear() でハイライトを消せます

## よくある組み合わせ

### ピッカーの選択を別ウィジェットへ渡す

```
connect(picker, &ObjectPicker::objectPicked,
        propertyEditor, &PropertyEditor::setInspectedObject);
```

### ツリー選択とピッカー側の選択状態をそろえる

```
connect(treeDialog->objectTreeWidget(), &ObjectTreeWidget::objectActivated,
        this, [picker](QObject *targetObject) {
            picker->setSelectedObject(targetObject);
        });
```

## 注意点

- ObjectPicker で直接選べるのは可視の QWidget です
- QLayout は画面上から直接ピックできないため、ツリー側から選択してください
- hoveredObjectChanged は一時候補の通知であり、確定選択ではありません
- キャンセル終了では objectPicked は emit されません
- 配布物には Qt 本体は含まれていません
- Qt の導入場所によっては CMAKE\_PREFIX\_PATH の指定が必要です